ASSIGNMENT: FROM GIT TO GOLD

Data Engineering · Pipeline Orchestration

Student details	Information
Name	Maddula Hemanth Kumar Reddy
Roll number	23BCS153
Programme	B.Tech – Computer Science and Engineering
Institute	PDPM IIITDM Jabalpur
GitHub repository	https://github.com/maddulahemanthkumarreddy-afk/freshcart-pipelines
Environment	Python 3.12.10 • Mage AI 0.9.80 • pika (Python RabbitMQ client)

Report scope

This report answers the numbered questions in Parts A–D of the "From Git to Gold" assignment and documents the optional Bonus +10 RabbitMQ-to-FreshCart lakehouse integration. It records observed notebook, test, Mage, RabbitMQ and Snowflake results. The architecture diagram and reflection are included for Part E. Evidence screenshots are placed in a separate appendix; the appendix is excluded from the eight-page main-report limit.

Repository

https://github.com/maddulahemanthkumarreddy-afk/freshcart-pipelines

Part A – Git and CI/CD

A1. Create the repository

The GitHub repository is named freshcart-pipelines. Its project structure includes the README and requirements files, the FreshCart notebook under elt/, the Mage project under iris_project/, RabbitMQ scripts under stream/, pytest tests under tests/, and the GitHub Actions workflow under .github/workflows/ci.yml. The .gitignore is configured to exclude sensitive and generated files such as .env, CSV outputs, pickle files, Mage logs and __pycache__.

Repository URL: https://github.com/maddulahemanthkumarreddy-afk/freshcart-pipelines

Commit history evidence: the supplied workflow evidence identifies the initial project setup and the Part A implementation, intentional-failure and fix commits. The assignment asks for at least five meaningful commits and the complete output of git log --oneline. Only the following four commit descriptions/IDs were recoverable from the available notes; do not treat this excerpt as proof of the five-commit requirement:

- bd0ecc9 – Initial project setup
- 0101efb – feat(part-a): implement prepare_data block, test_prepare_data, and GitHub Actions CI workflow
- 700a375 – chore: improve gitignore for local development files
- 0e117a8 – chore: exclude temporary Mage scaffold and cache files

Additional commits: 5355dd6 – feat(part-b): add FreshCart Snowflake ELT notebook; 165e74a – feat(part-c): add Mage Iris classification pipeline and evaluation; 423429e – feat(part-d): add RabbitMQ dispatcher and ACK NACK drivers; 1439dc3 – fix(part-b): add future order date quality validation. The final history has eight commits.

A2. Where does each command act?

Command	Where the change happens	What exactly happens
git init	Working directory / .git	Creates a new local Git repository and its metadata.
git add train.py	Staging area	Stages the current version of train.py for the next commit.
git commit -m "msg"	Local repository	Saves the staged snapshot as a local commit with a message.
git remote add origin URL	Local repository configuration	Registers the remote repository URL under the name origin.

git push -u origin main	Remote repository (GitHub)	Uploads local commits on main and sets the upstream tracking branch.
git pull origin main	Local repository	Fetches the remote main branch and integrates its changes into the current branch.

Can a teammate see code immediately after git commit? No. A commit is stored in the local repository only. The teammate can see it on GitHub after the commit is pushed to the remote repository.

A3. A small CI workflow

The pytest test imports the prepare_data transformation and checks that the Iris dataset is split into 120 training rows and 30 test rows (an 80/20 split of 150 records). The GitHub Actions workflow checks out the repository, sets up Python 3.12, installs requirements.txt and runs pytest on pushes.

CI experiment	Observed result	Meaning
Initial workflow run	1 test passed	The 80/20 split satisfied the test.
Intentional 70/30 change	Test failed: expected 120 training rows, observed 105	The test detected the incorrect split.
Restore 80/20 split	1 test passed	The fix restored the expected 120/30 split.

The red run is intentional evidence of the test working, not an unresolved CI failure. The subsequent fix run is green.

Part B – FreshCart ELT on Snowflake

B1. Run the baseline

The notebook setup completed and the FreshCart pipeline ran through extraction, loading, transformation and quality checks. The observed baseline results are:

Check	Observed result
Bronze raw order rows	8 (6 web + 2 mobile)
Silver clean orders	6
Quarantine rows	2: ORD-1004 and ORD-1006
Orders after deduplication	4
Gold result	4 orders; revenue ₹7,785.00; 126 loyalty points
Share of rows rejected	25% (2 of 8 checked rows)

B2. Trace one record by hand

ORD-1003 appears twice in Bronze: the website record contains customer-style fields and has no amount, while the mobile record contains order-style fields, has amount ₹2,100 and a later updated_at timestamp. SQL_10_ORDERS_CLEAN maps the source-specific JSON attributes into a common set of columns in silver.orders_checked, while retaining source and ingestion metadata.

SQL_20_ORDERS_DEDUP uses QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1. The mobile version survives because its updated_at is later. Its INR amount is ₹2,100 × 1.00 = ₹2,100. The points calculation is FLOOR(2,100 / 100) × 3 (Platinum multiplier) = 21 × 3 = 63 loyalty points.

For ORD-1005, Priya receives Gold-tier points because the order date matches the Gold validity interval in dim.customers_scd2. The point-in-time join uses the customer key and the order date between valid_from and valid_to. ORD-1001 falls in Priya's Silver interval: FLOOR(1,250 / 100) × 1 = 12 points. ORD-1005 uses Gold: FLOOR(899 / 100) × 2 = 16 points.

B3. Add a validation rule

The cleaning validation adds unsupported_currency to the reasons array when a currency is not INR or USD. ORD-1006 was already invalid because of bad_email and bad_pincode; after the new rule, unsupported_currency is added as another rejection reason. The Gold total does not change because the row remains quarantined and does not enter the clean, deduplicated or Gold fact data. The quarantine insertion uses a NOT EXISTS guard on the source/order identity so the same rejected record is not inserted again on a rerun.

B4. Change history and idempotency

A CRM event changes Arjun (CUST-02, arjun.k@yahoo.in) to Gold effective 2026-08-15. The observed SCD2 history has two rows for CUST-02: Silver ending 2026-08-14 and current Gold beginning 2026-08-15. ORD-1002 is assigned Gold and earns 70 points. The total Gold loyalty points after the change are 161. The idempotency check showed 4 Gold rows and 0 unchanged orders republished / 0 changed row hashes.

ORD-1002 calculation: USD 42.50 × FX rate 83.20 = ₹3,536.00. Gold points = FLOOR(3,536 / 100) × 2 = 35 × 2 = 70.

Bronze is append-oriented, so each extraction run can add another batch of raw source records. The Silver deduplication step selects the latest record per order. In Gold, row_hash represents the business attributes; the MERGE compares the incoming row with the stored row and inserts or updates only when the business data is new or changed. Thus, an unchanged rerun does not duplicate the Gold facts or republish unchanged rows.

B5. Add one data-quality check

The selected B5 check is “future order dates” in silver.orders_dedup. It returned 0 / OK on the baseline data. A temporary change of ORD-1001 to 2099-01-01 inside a transaction made the check return 1 / PROBLEM. The transaction was rolled back and ORD-1001 restored to 2026-09-01; the check returned 0 / OK again. Other displayed checks returned zero and the rejected-row share remained 25%.

This satisfies the assignment option to check for future order_date values in silver.orders_dedup. The unsupported-currency check is retained as an additional validation.

ELT is suitable for FreshCart because raw web/mobile records can be landed first in Bronze and retained, while Snowflake performs cleaning, validation, deduplication and enrichment using scalable SQL. This makes transformation logic easier to change without losing source data. ETL can still be preferable when sensitive information must be masked or filtered before it is transferred to the warehouse, or when the destination has limited transformation capability.

Part C – ML pipeline with Mage

C1. Draw and run the pipeline

The initial Mage graph is load_iris (data_loader) → prepare_data (transformer) → train_model (transformer) → save_results (data_exporter). Each block has an attached @test. The command mage run iris_project iris_classifier --test completed successfully. It loaded 150 Iris records, used a 120/30 train/test split, trained the default depth-3 Decision Tree and saved 30 predictions. The observed test accuracy was 0.9667 (96.67%).

C2. Tests as a gate

max_depth	Test accuracy	test_accuracy passed?	CSV created?
1	66.67%	No	No
2	93.33%	Yes	Yes
3	96.67%	Yes	Yes
4	93.33%	Yes	Yes
5	93.33%	Yes	Yes

The train_model @test requires accuracy ≥ 0.90. At max_depth=1, accuracy is 0.6667, so the test fails and Mage does not execute the downstream save_results exporter; therefore iris_predictions.csv is not created. At depths 2–5, the test passes and the CSV is created with 30 predictions. This gate is safer than saving a poor result and checking it afterwards because it prevents invalid output from becoming a downstream artifact.

C3. Add the evaluation branch (5 marks)

The edited metadata.yaml configures train_model with two downstream blocks: save_results and evaluate_model. The evaluation block prints the confusion matrix and classification metrics. With max_depth=3, the observed confusion matrix (rows = actual; columns = predicted) is:

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	0	10

Class	Precision	Recall	F1-score	Support
Setosa	1.0000	1.0000	1.0000	10
Versicolor	1.0000	0.9000	0.9474	10
Virginica	0.9091	1.0000	0.9524	10
Accuracy	0.9667	0.9667	0.9667	30

The Mage UI graph screenshot shows the branch from train_model to save_results and evaluate_model.

C4. Your own block

An additional exporter named export_metrics was added downstream of evaluate_model. It writes the evaluation metrics DataFrame to iris_metrics.csv. Its @test verifies that the output file exists, has at least five metric rows, and includes class, precision, recall and f1-score columns. The pipeline run and metrics CSV screenshot provide evidence of this extra block.

Part D – Food-order stream with RabbitMQ

D1. Routing on paper

In a RabbitMQ topic exchange, * matches exactly one dot-separated word and # matches zero or more words.

#	Routing key	Receiving queues
1	bangalore.koramangala.food	Ravi, Meena, Control
2	bangalore.indiranagar.ride	Priya, Arjun, Control
3	bangalore.whitefield.ride	Arjun, Control
4	chennai.tnagar.food	Karthik
5	chennai.whitefield.ride	Dropped
6	bangalore.koramangala	Control only

For key 6, bangalore.koramangala has only two words. The driver bindings ending in * or requiring bangalore.*.ride / bangalore.*.food need three words and therefore do not match. Control's bangalore.# matches any number of words after bangalore, including zero, so only Control receives it.

D2. Build it

stream/dispatcher.py connects to the hosted RabbitMQ broker using the URL loaded from the project-root .env file, declares the durable topic exchange rapido-dispatch, and publishes JSON messages every two seconds. Each message includes id, city, zone, type and a UTC timestamp; the routing key is city.zone.type. stream/driver.py accepts a driver name and binding pattern, declares a private queue, binds it to the exchange, prints each received message and acknowledges it. The submitted console evidence shows the dispatcher and three drivers—Arjun, Priya and Ravi—receiving matching messages.

D3. ACK and NACK

The dispatcher was changed to omit the zone field once every ten messages. The driver callback checks for zone. If it is missing, the driver sends basic_nack(requeue=False), causing the malformed message to be discarded (or routed to a configured dead-letter exchange, if one is configured). For a valid message, it sends basic_ack. The screenshots show good messages receiving ACK and bad messages producing "BAD MESSAGE: zone field missing" followed by "Sending NACK – message will be discarded".

If the driver crashes before sending an ACK, the delivery remains unacknowledged. When the consumer channel or connection closes, RabbitMQ can requeue the unacknowledged delivery for another consumer or a later consumer

session. If requeue=True is used for a permanently malformed message, it may be delivered repeatedly, creating a poison-message loop and consuming resources. requeue=False prevents that repeated redelivery.

Optional Bonus – RabbitMQ to FreshCart lakehouse

Implementation

A standalone consumer is provided at stream/bonus_consumer.py so the streaming ingestion can be run outside the notebook. It declares the durable topic exchange rapido-dispatch, declares and binds the freshcart-lakehouse-bonus queue with freshcart.orders.*, collects valid FreshCart-style order events for a configurable window (60 seconds by default), and writes a gzip-compressed JSON Lines landing file using the same source/run_id/extracted_at/data envelope as write_landing(). Messages are ACKed only after the landing file is written successfully; malformed messages are NACKed without requeue. The optional --load-bronze flag uploads the landing file to the existing Bronze stage and copies it into bronze.orders_raw. The existing notebook SQL then performs Silver cleaning, deduplication, customer/FX/geography enrichment and the Gold MERGE.

Observed execution and results

The publisher sent six events, ORD-STREAM-0001 through ORD-STREAM-0006, to rapido-dispatch using routing key freshcart.orders.created. The consumer collected all six during a 60-second window, acknowledged all six after landing, and created freshcart_landing/web/web__BONUS_20261107_103348.json.gz. Bronze orders_raw increased from 16 to 22 rows (+6). All six new orders appeared in silver.orders_clean and silver.orders_dedup. Each was INR 500.00 for pincode 400050. After the existing customer, FX and pincode enrichment and SQL_40_GOLD, all six appeared in gold.fct_orders with 15 loyalty points each.

Gold totals before the bonus load were 4 rows, ₹7,785.00 revenue and 161 loyalty points. After the six streamed orders, Gold contained 10 rows, ₹10,785.00 revenue and 251 loyalty points. The six bonus orders contributed ₹3,000.00 and 90 points. A second execution of the Gold MERGE left the totals unchanged at 10 / ₹10,785.00 / 251, confirming the Gold MERGE is idempotent for the replayed Silver data. This test verifies replay safety at the Gold MERGE step; it does not claim that replaying the entire RabbitMQ-to-Bronze ingestion batch is idempotent.

Reproduction: install pika, python-dotenv and snowflake-connector-python; keep RABBITMQ_URL and Snowflake credentials in the root .env file (never commit .env); run python stream/bonus_publisher.py in one terminal and python stream/bonus_consumer.py --duration 60 --load-bronze in another. Set the Snowflake session variable SET RUN_ID = '<RUN_ID>' before running SQL_10_ORDERS_CLEAN, then run SQL_20_ORDERS_DEDUP, SQL_30_CUSTOMERS_SCD2, SQL_31_FX_DAILY, SQL_32_PINCODE_GEO and SQL_40_GOLD from elt/freshcart_elt.ipynb.

Part E – Integration and reflection

Integrated architecture

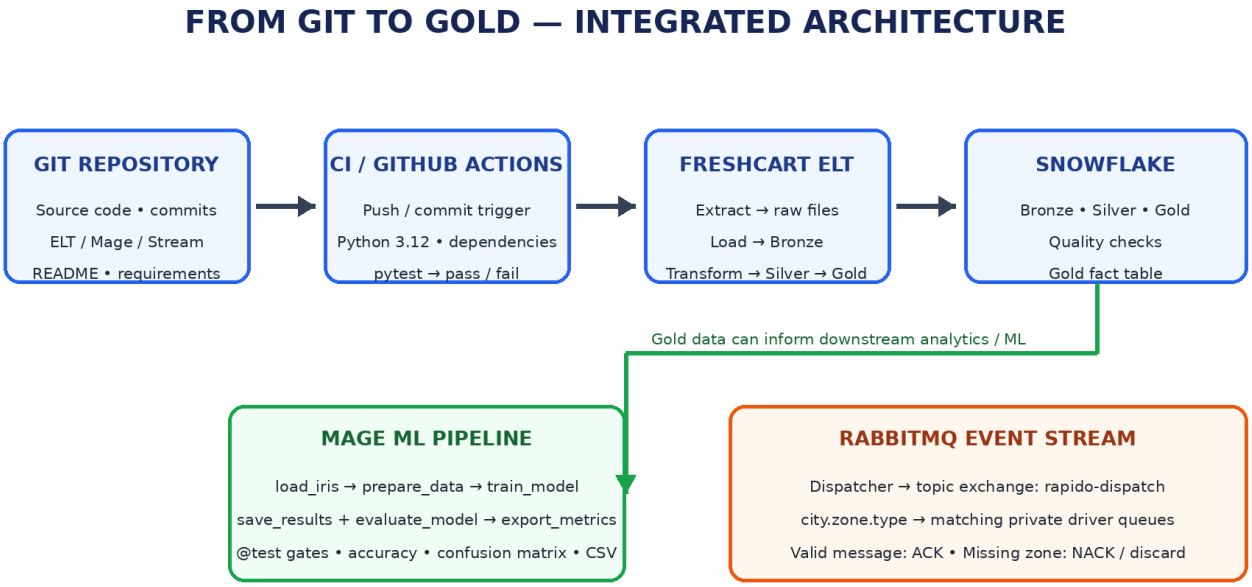


Figure 1. Relationship between source control, automated testing, warehouse ELT, ML orchestration and event streaming.

Half-page reflection

One issue encountered during the streaming setup was that Docker was not available in the Windows PowerShell environment: the docker command was not recognized. This showed that a local Docker-based RabbitMQ setup could not be used without installing and configuring Docker Desktop. I avoided blocking the assignment by creating a hosted RabbitMQ instance through CloudAMQP and storing its AMQPS connection URL in the project-root .env file, which is excluded by .gitignore. I installed the pika client and python-dotenv in the active virtual environment and tested the broker connection before starting the dispatcher and multiple drivers in separate VS Code terminals. After the basic routing worked, I deliberately removed the zone field from every tenth published message. The driver output confirmed that valid messages were acknowledged and malformed messages were rejected with basic_nack(requeue=False). This debugging process taught me to isolate broker availability, credentials, topic routing and acknowledgement handling rather than treating the stream as one undifferentiated failure.

Conclusion

The project demonstrates a connected data-engineering workflow: Git records source changes, CI validates code on push, FreshCart ELT produces curated Gold facts, Mage orchestrates tested ML blocks, and RabbitMQ streams order events into the same Bronze-to-Gold lakehouse. The bonus run confirms six streamed orders reach Gold and that the Gold MERGE does not double-count unchanged rows when replayed.

Final pre-submission checks

- The report records the eight meaningful commits and the final pushed commit 1439dc3. The repository and CI evidence are included in Appendix A.
- B5 is completed: the future-order-date quality check was tested with a temporary future-dated record inside a transaction, the failure was observed, and the transaction was rolled back. The restored dataset passed all checks.
- Confirm the notebook, Mage project, stream scripts and report are committed and the repository is accessible to the instructor.